

MA SOFT TECH SOLUTIONS

DEVOPS INTERNSHIP – PRACTICAL PROJECT TESTING TASK

Project Title: Production-Ready E-Commerce Application – DevOps Deployment, Monitoring & Reliability

Project Objective

The interns will take a simple E-Commerce web/API application and make it production-ready through the complete DevOps lifecycle. The project must demonstrate source control, Linux administration, containerization, CI/CD, deployment, monitoring, security, troubleshooting, rollback, and recovery.

1. Core Project Requirements

- Create and manage a Git/GitHub repository with meaningful commits, branches, .gitignore, pull request and merge workflow.
- Prepare a Linux-based deployment environment and configure required users, permissions, processes, ports and environment variables.
- Containerize the application using Docker. Include a proper Dockerfile, .dockerignore, port configuration, environment variables and health check.
- Use Docker Compose where appropriate for application and supporting services.
- Build an automated CI/CD pipeline: Git Push → Checkout → Build → Test → Docker Build → Security Check → Deploy → Health Check.
- Deploy the application to a Linux/Cloud VM or a realistic local production-like environment.
- Configure networking, firewall/security-group concepts, required ports and Nginx/reverse proxy where required.
- Implement application health checks and verify the application after deployment.
- Set up Prometheus and Grafana to monitor CPU, RAM, Disk, Network and application health.
- Configure logging and alerts for important failures and resource thresholds.
- Demonstrate rollback and recovery after a failed deployment.
- Maintain clear technical documentation for setup, deployment, troubleshooting and recovery.

2. Team Structure & Responsibilities

Person 1 – DevOps & Infrastructure Intern	Person 2 – DevOps & Monitoring Intern
• Linux/server setup and administration • Git/GitHub workflow • Docker & Docker Compose • CI/CD pipeline • Networking, ports & Nginx • Environment/configuration • Application deployment • Rollback & recovery • Infrastructure documentation	• Prometheus setup • Grafana dashboard • CPU/RAM/Disk/Network monitoring • Application health checks • Logs & alerting • Security scanning/basic DevSecOps • Failure testing • Performance monitoring • Incident troubleshooting & verification

3. Mandatory DevOps Activities

Area	Expected Work
Git & GitHub	Repository, branches, meaningful commits, .gitignore, pull request and merge.
Linux	Users/permissions, processes, ports, environment variables and logs.
Docker	Dockerfile, .dockerignore, image, container, ports, environment variables and health check.
CI/CD	Automated build, testing, Docker build, validation and deployment.
Testing	Application/API tests, container validation and post-deployment health testing.
Infrastructure	Server/VM, networking, firewall/security-group concepts and reverse proxy.

Area	Expected Work
Monitoring	Prometheus + Grafana dashboards for system and application metrics.
Logging	Application/container logs and error investigation.
Security	No committed secrets, least-privilege access and vulnerability/security scanning.
Reliability	Health checks, alerts, failure detection, recovery and rollback.

4. Failure Injection & Troubleshooting Test

The supervisor should intentionally introduce one or more failures. Interns must investigate logically instead of simply restarting services. For every failure, they must document: Failure → Evidence → Hypothesis → Investigation → Root Cause → Fix → Rebuild/Redeploy → Retest → Verification.

- Wrong application/container port
- Invalid environment variable or configuration
- Broken Docker configuration or container startup failure
- Failed application/CI test
- Application crash or unhealthy container
- Health-check failure
- High CPU or disk-usage condition
- CI/CD pipeline failure
- Permission or access problem
- Monitoring/alert configuration failure

5. Final Deployment & Incident Scenario

Scenario: A new application version has been deployed. The container appears to be running, but users cannot access the application. The interns have access to the server terminal, Docker information, CI/CD logs and monitoring dashboard.

- Identify the problem using a logical troubleshooting sequence.
- Check application status, container status, ports, logs, health check, configuration and networking.
- Identify and explain the root cause.
- Apply the correct fix without unnecessarily changing unrelated components.
- Rebuild/redeploy the application if required.
- Verify application availability and health.
- Demonstrate monitoring/alerting evidence.
- Explain how the issue could be prevented in the future.

6. Required Deliverables

- Complete Git repository with clean structure and meaningful commit history.
- Dockerfile and Docker-related configuration.
- CI/CD pipeline configuration.
- Deployment configuration and environment-variable documentation.
- Prometheus configuration and Grafana dashboard.
- Monitoring and alert rules.
- Testing evidence and screenshots.
- Failure-injection and troubleshooting report.
- Deployment/rollback documentation.
- Final architecture diagram.
- Final project walkthrough/demo.

7. Final Evaluation Focus

Area	What Will Be Checked
Technical Implementation	Working deployment, Docker, CI/CD and infrastructure.

Area	What Will Be Checked
Troubleshooting	Ability to investigate, identify root cause and recover from failures.
Monitoring	Useful metrics, dashboards, logs and alerts.
Security	Secrets management, permissions and basic vulnerability controls.
Reliability	Health checks, rollback and recovery process.
Documentation	Clear setup, commands, architecture and troubleshooting evidence.
Teamwork	Clear ownership, coordination and understanding of the complete project.

Expected Outcome: By the end of the task, both interns should be able to explain and demonstrate the complete DevOps workflow from Git commit to deployment, monitoring, failure handling and recovery.